

Roboflow와 YOLOv10을 활용한 Custom 객체 탐지 가이드

Can, Cup, Pet 3종 객체 분류 모델 개발

가이드 개요

본 가이드는 Roboflow를 활용하여 Can(캔), Cup(컵), Pet(애완동물) 3종류의 객체를 탐지하는 YOLOv10 Custom 모델을 개발하는 전체 과정을 단계별로 안내합니다.

학습 목표

- Roboflow를 이용한 커스텀 데이터셋 준비 및 전처리
- YOLOv10 모델을 활용한 커스텀 객체 탐지 모델 학습
- 학습된 모델로 이미지 분류 및 실시간 라이브 스트림 테스트

목차

-  [1단계: Custom 데이터 준비 및 Roboflow 전처리](#)
 - [데이터셋 준비 및 업로드](#)
 - [Roboflow 전처리 및 증강](#)
 - [데이터셋 내보내기](#)
-  [2단계: Custom 데이터로 YOLOv10 학습](#)
 - [학습 환경 준비](#)
 - [데이터셋 배치](#)
 - [학습 명령어 실행](#)
 - [학습 결과 확인](#)
-  [3단계: Custom 모델로 이미지 분류 테스트](#)
-  [4단계: Custom 모델로 라이브 스트림 분류 테스트](#)

1단계: Custom 데이터 준비 및 Roboflow 전처리

Custom 데이터를 YOLOv10 학습에 적합한 형태로 변환하기 위해 Roboflow를 사용합니다.

1. 데이터셋 준비 및 업로드

데이터 수집 요구사항

- **Can(캔):** 다양한 브랜드의 음료수 캔 이미지 최소 200장
 - **Cup(컵):** 머그컵, 종이컵, 플라스틱컵 등 다양한 컵 이미지 최소 200장
 - **Pet(애완동물):** 개, 고양이 등 애완동물 이미지 최소 200장
-  **팁:** 각 클래스별로 다양한 각도, 조명, 배경에서 촬영된 이미지를 사용하면 모델 성능이 향상됩니다.

1. **이미지 수집:** Can, Cup, Pet 객체가 포함된 충분한 양의 이미지를 수집합니다.
2. **어노테이션 (Annotation):** 각 이미지에서 객체의 위치에 바운딩 박스(Bounding Box)를 그리고 해당 객체의 클래스를 지정합니다.
3. **Roboflow 프로젝트 생성:**

- Roboflow 웹사이트에 가입/로그인하고 새로운 프로젝트를 생성합니다.
- Project Type은 **Object Detection**으로 설정합니다.
- 클래스 설정: can, cup, pet
- 어노테이션된 이미지들을 Roboflow에 업로드합니다.

2. Roboflow 전처리 및 증강 (Preprocessing & Augmentation)

Roboflow는 모델 성능 향상을 위해 유용한 전처리와 데이터 증강 기능을 제공합니다.

Preprocessing (전처리)

- **Resize:** YOLOv10이 일반적으로 사용하는 해상도(예: 640x640)로 이미지 크기를 조정합니다.
- **Auto-Orient:** 이미지 방향을 자동으로 교정합니다.

Augmentation (데이터 증강)

데이터 다양성을 늘려 모델의 일반화 성능을 높입니다.

- **Rotation:** 15도 범위 내 회전
- **Flip:** 50% 확률로 좌우 반전
- **Brightness:** $\pm 20\%$ 밝기 조절
- **Shear:** ± 10 도 기울기 조절

주의사항

Pet 클래스의 경우 좌우 반전 시 자연스럽지 않을 수 있으므로, 증강 설정을 신중하게 선택하세요.

3. 데이터셋 내보내기 (Export)

1. **Generate**를 클릭하여 전처리 및 증강이 적용된 새로운 버전의 데이터셋을 생성합니다.
2. **Export**를 클릭하고 다음 설정을 선택합니다:
 - **Format:** YOLOv5 PyTorch (YOLOv10은 YOLOv5/v8과 유사한 데이터 형식을 사용합니다.)
 - **Show Download Code:** 체크합니다.
3. 다운로드 창에 나타나는 코드를 복사하거나, download.zip 파일을 다운로드합니다.

2단계: Custom 데이터로 YOLOv10 학습 (Training)

Roboflow에서 다운로드한 데이터를 사용하여 Conda 환경에서 학습을 시작합니다.

1. 학습 환경 준비

이전 단계에서 설치한 yolov10_env Conda 환경이 활성화되어 있어야 합니다.

```
conda activate yolov10_env
```

2. 데이터셋 배치

Roboflow에서 다운로드한 압축 파일(download.zip)을 yolov10_project 폴더에 압축 해제합니다.

압축을 해제하면 보통 다음과 같은 구조의 폴더가 생깁니다. (data.yaml 파일이 중요합니다.)

```
/yolov10_project
└─ /train/
```

```
└─ /valid/  
└─ /test/  
└─ data.yaml
```

data.yaml 파일 예시

이 파일에는 클래스 정보와 데이터 경로가 포함되어 있습니다:

```
train: ../train/images  
val: ../valid/images  
test: ../test/images  
nc: 3  
names: ['can', 'cup', 'pet']
```

3. 학습 명령어 실행

YOLOv10은 ultralytics 라이브러리를 통해 간단히 학습을 실행할 수 있습니다. 학습할 때 사용할 기본 모델(예: yolov10n.pt)이 필요합니다.

```
# YOLOv10 Nano 모델을 사용해 50 Epoch 동안 학습하는 명령어  
# 'yolov10n.pt'는 모델 가중치 파일이며, 'data.yaml'은 데이터셋 경로와 클래스 정보 파일입니다.  
python -m ultralytics yolo train \  
    model=yolov10n.pt \  
    data=data.yaml \  
    epochs=50 \  
    imgsz=640 \  
    batch=16 \  
    name=yolov10_can_cup_pet_train
```

학습 파라미터 설명

- **model:** 사전 학습된 기본 모델 가중치 (Transfer Learning)
- **data:** Roboflow에서 제공한 데이터셋 설정 파일
- **epochs:** 전체 데이터셋을 몇 번 반복 학습할지 설정 (50~100 권장)
- **imgsz:** 이미지 크기 (640x640)
- **batch:** 한 번에 처리할 이미지 수 (PC 성능에 따라 조절: 8, 16, 32)
- **name:** 학습 결과 저장 폴더 이름

하드웨어 권장사항

- GPU: NVIDIA RTX 3060 이상 권장
- RAM: 16GB 이상
- 학습 시간: 약 2-4시간 (데이터 크기에 따라 차이)

4. 학습 결과 확인

학습이 완료되면, 학습 결과 파일과 최종 모델 가중치 파일은 runs/detect/yolov10_can_cup_pet_train/weights/best.pt 경로에 저장됩니다. 이 best.pt 파일을 다음 테스트 단계에서 사용합니다.

학습 완료 확인사항

- best.pt 파일 존재 확인
- 학습 로그에서 mAP(mean Average Precision) 값 확인
- results.png에서 학습 곡선 확인



3단계: Custom 모델로 이미지 분류 테스트 (Inference)

학습된 best.pt 모델을 사용하여 Can, Cup, Pet 객체를 탐지하는 파이썬 코드입니다.

1. 환경 설정

- yolov10_project 폴더에 다음 코드를 **custom_image_classify.py**로 저장합니다.
- 학습된 모델 파일(best.pt)을 사용할 수 있는 경로에 두거나, 코드에서 경로를 수정하세요.
- 테스트할 이미지를 ./images/test_img.jpg 경로에 준비합니다.

2. 이미지 분류 파이썬 코드 (custom_image_classify.py)

이 코드는 학습된 모델로 이미지를 탐지하고, Ultralytics의 기본 기능을 사용하여 결과를 저장합니다.

```
from ultralytics import YOLO
import os

# --- 설정 (Configuration) ---
# 1. 학습 완료된 Custom 모델 가중치 파일 경로
# 'best.pt'는 2단계에서 학습 완료 후 저장된 파일입니다.
custom_model_path = './runs/detect/yolov10_can_cup_pet_train/weights/best.pt'

# 2. 테스트할 이미지 파일 경로
image_path = './images/test_img.jpg'

if not os.path.exists(custom_model_path):
    print(f"❌ 오류: Custom 모델 파일 '{custom_model_path}'을 찾을 수 없습니다. 경로를 확인하세요.")
    exit()
if not os.path.exists(image_path):
    print(f"❌ 오류: 이미지 파일 '{image_path}'을 찾을 수 없습니다. 경로를 확인하세요.")
    exit()

# --- 2. 모델 로드 및 추론 실행 ---
try:
    model = YOLO(custom_model_path)
    print(f"✅ Custom 모델 로드 완료: {custom_model_path}")
except Exception as e:
    print(f"❌ 모델 로드 중 오류 발생: {e}")
    exit()

print(f"🚀 이미지 추론 시작: {image_path}")

# 추론 실행
results = model.predict(
    source=image_path,
    conf=0.25,      # 탐지 신뢰도 임계값
    save=True,      # 결과 이미지를 runs/detect/predict 폴더에 저장
    show=True,      # 결과를 화면에 표시 (OpenCV 필요)
    name='can_cup_pet_result' # 결과 저장 폴더 이름 지정
)

# --- 3. 결과 출력 ---
if results:
    # results[0]은 첫 번째 이미지의 결과
    num_detections = len(results[0].boxes)
    print(f"\n✅ 검출된 총 객체 수: {num_detections}")

    # 각 검출된 객체의 클래스와 신뢰도 출력
    for i, box in enumerate(results[0].boxes):
        class_id = int(box.cls[0])
        confidence = float(box.conf[0])
        class_name = model.names[class_id]
        print(f"  객체 {i+1}: {class_name} (신뢰도: {confidence:.2f})")

    print(f"🎉 결과 이미지는 'runs/detect/can_cup_pet_result' 폴더에 저장되었습니다.")
else:
    print(f"❌ 검출된 객체가 없습니다.")
```

3. 실행

python custom image classify.py

예상 결과

- Can 객체 탐지 시: "can (신뢰도: 0.85)"
- Cup 객체 탐지 시: "cup (신뢰도: 0.92)"
- Pet 객체 탐지 시: "pet (신뢰도: 0.78)"
- 결과 이미지에 바운딩 박스와 라벨이 표시됩니다.



4단계: Custom 모델로 라이브 스트림 분류 테스트

학습된 best.pt 모델을 사용하여 웹캠을 통해 실시간으로 Can, Cup, Pet 객체를 탐지하는 파이썬 코드입니다.

1. 환경 설정

- yolov10_project 폴더에 다음 코드를 **custom_livestream_classify.py**로 저장합니다.
- 이전 코드와 마찬가지로 best.pt 파일 경로를 확인합니다.
- 웹캠이 연결되어 있는지 확인합니다.

2. 라이브 스트림 분류 파이썬 코드 (custom_livestream_classify.py)

이 코드는 model.predict 메소드를 사용하여 웹캠 비디오 스트림을 처리하며, OpenCV를 통해 실시간으로 결과를 보여줍니다.

```
from ultralytics import YOLO
import cv2
import os

# --- 1. 설정 (Configuration) ---
# 학습 완료된 Custom 모델 가중치 파일 경로
custom_model_path = './runs/detect/yolov10_can_cup_pet_train/weights/best.pt'
webcam_index = 0 # 기본 웹캠 (보통 0)

if not os.path.exists(custom_model_path):
    print(f"❌ 오류: Custom 모델 파일 '{custom_model_path}'을 찾을 수 없습니다. 경로를 확인하세요.")
    exit()

# --- 2. 모델 로드 및 웹캠 초기화 ---
try:
    model = YOLO(custom_model_path)
    print(f"✅ Custom 모델 로드 완료: {custom_model_path}")
    print(f"📦 탐지 가능한 클래스: {model.names}")
except Exception as e:
    print(f"❌ 모델 로드 중 오류 발생: {e}")
    exit()

# 웹캠 캡처 객체 초기화
cap = cv2.VideoCapture(webcam_index)
if not cap.isOpened():
    print(f"❌ 오류: 카메라를 열 수 없습니다. 카메라가 연결되어 있는지 또는 인덱스가 올바른지 확인하세요.")
    exit()

# 웹캠 해상도 설정 (선택사항)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

# --- 3. 실시간 프레임 처리 루프 ---
print(f"🚀 웹캠 실시간 Can, Cup, Pet 객체 검출을 시작합니다.")
print(f"📱 사용법: 'q' 키를 누르면 종료됩니다.")
print(f"🎯 팁: 캔, 컵, 애완동물을 카메라 앞에 보여주세요!")

while cap.isOpened():
    success, frame = cap.read()

    if not success:
        print("프레임을 받을 수 없습니다 (스트림 끝?). 종료합니다.")
```

```

break

# 추론 실행 (save=False, show=False 설정)
results = model.predict(
    source=frame,
    conf=0.25,          # 탐지 신뢰도 임계값
    imgsz=640,
    verbose=False,      # 출력 로그 최소화
    show=False,         # OpenCV로 수동 표시
    stream=True         # 실시간 스트림 처리를 최적화
)

# 결과 프레임 얻기 (Ultralytics가 바운딩 박스를 그려준 프레임)
# stream=True일 때 results는 제너레이터이므로 반복
annotated_frame = frame
for result in results:
    # result.plot()을 사용하여 Ultralytics가 결과를 프레임에 그려줍니다.
    annotated_frame = result.plot()

    # 콘솔에 탐지 결과 출력 (선택사항)
    if len(result.bboxes) > 0:
        detected_objects = []
        for box in result.bboxes:
            class_id = int(box.cls[0])
            confidence = float(box.conf[0])
            class_name = model.names[class_id]
            detected_objects.append(f"{class_name}({confidence:.2f})")
        print(f"🎯 탐지된 객체: {' '.join(detected_objects)}")

# 화면에 프레임 표시
cv2.imshow('YOLOv10 Can-Cup-Pet Live Detection', annotated_frame)

# 'q' 키를 누르면 루프 종료
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# --- 4. 종료 및 자원 해제 ---
cap.release()
cv2.destroyAllWindows()
print("\n🎉 실시간 Can, Cup, Pet 객체 검출을 종료합니다.")

```

3. 실행

python custom_livestream_classify.py

🎮 실시간 테스트 가이드

1. **Can 테스트:** 음료수 캔을 카메라 앞에 보여주세요
2. **Cup 테스트:** 머그컵, 종이컵 등을 카메라 앞에 보여주세요
3. **Pet 테스트:** 애완동물이나 애완동물 사진을 카메라 앞에 보여주세요
4. 각 객체 주변에 바운딩 박스와 클래스명이 실시간으로 표시됩니다

🔧 성능 최적화 팁

- conf 값을 0.5로 높이면 더 확실한 객체만 탐지합니다
- imgsz를 320으로 낮추면 처리 속도가 빨라집니다
- GPU가 없는 경우 CPU 모드로 자동 실행됩니다

🎉 완성!

이제 학습된 Custom YOLOv10 모델을 사용하여 Can, Cup, Pet 객체를 이미지와 실시간 웹캠 스트림에서 성공적으로 탐지할 수 있습니다.

✨ 추가 개선 방향

- 데이터셋 크기를 늘려 모델 성능 향상

- 다양한 YOLOv10 모델 크기 테스트 (n, s, m, l, x)
- 하이퍼파라미터 튜닝으로 최적화
- 추가 객체 클래스 확장 가능

문제 해결

모델 학습이나 테스트 중 문제가 발생하면 다음을 확인해보세요:

- CUDA/GPU 드라이버 설치 상태
- 데이터셋 경로와 data.yaml 파일 설정
- 충분한 디스크 공간과 메모리
- Python 패키지 버전 호환성